

Best Sources to Learn Algorithms, Big O Notation, and Complexity

Books (Best for Deep Understanding)

1. Grokking Algorithms by Aditya Bhargava

- Beginner-friendly with visuals and code (in Python).
- Perfect intro to Big O, recursion, searching, sorting, graphs.

2. Introduction to Algorithms by Cormen, Leiserson, Rivest, and Stein (CLRS)

- Industry-standard textbook.
- Great for intermediate to advanced learners; used in CS degree programs.

3. Algorithms by Robert Sedgewick & Kevin Wayne

- Accompanies Princetons famous course (Java-focused).
- Strong emphasis on performance and complexity.

4. The Algorithm Design Manual by Steven S. Skiena

- Practical and theoretical mix.
- Includes a catalog of the most important algorithms.

Online Courses & Platforms

1. CS50: Introduction to Computer Science Harvard (edX)

- Excellent foundation.
- Covers algorithms, data structures, Big O, problem solving.

2. MIT OpenCourseWare Introduction to Algorithms

- Free, in-depth lectures from top professors.
- Closely follows the CLRS book.

3. Coursera Data Structures and Algorithms Specialization (UC San Diego)

- Structured into 6 courses.
- Includes programming assignments and complexity analysis.

4. Udemy Master the Coding Interview: Big O, Data Structures + Algorithms

- Taught by Andrei Neagoie.
- Very clear on Big O and practical coding.

Interactive Platforms

1. Visualgo

- Visualize sorting, trees, graphs, etc. with complexity labels.
- Great for understanding how algorithms work step-by-step.

2. Big-O Cheat Sheet

- Quick reference for complexities of common operations.
- Includes best/worst/average cases.

3. LeetCode

- Top choice for practicing algorithm problems.
- Each problem usually has discussions on Big O.

4. AlgoExpert (paid)

- Curated set of 160+ algorithm problems.
- Detailed solutions with complexity explanations.

5. GeeksforGeeks

- Huge library of tutorials and examples.
- Includes time & space complexity of all algorithms.

YouTube Channels

1. Abdul Bari

- Legendary for clear algorithm explanations.

2. William Fiset

- Deep dives into trees, graphs, complexity, and data structures.

3. Back To Back SWE

- Focus on algorithmic thinking and Big O analysis for coding interviews.

4. Tech With Tim

- Friendly and beginner-oriented algorithm series (Python).

Suggested Study Path

1. Start with Basics:

- Learn about arrays, linked lists, stacks, queues.
- Understand Big O: $O(1)$, $O(n)$, $O(\log n)$, $O(n^2)$.

2. Move to Intermediate:

- Learn sorting (quick, merge, heap), recursion, binary search.
- Explore trees, hash maps, graphs, and basic dynamic programming.

3. Analyze Complexity:

- Practice estimating time and space complexity.
- Compare algorithms by analyzing operations in code.

4. Practice Problems:

- Start easy problems on LeetCode or HackerRank.
- Gradually move to medium and hard as you master complexity.